==========
PowerShell Agent 实战避坑与原生执行指南 (PowerShell Agent Guide)
==========

1. 概述与核心定位

在自主编程智能体（AI Coding Agent）体系中，命令执行工具（run_command）是智能体感知工作区现状与验证工程成果的关键物理触手。在 Windows 宿主环境下，PowerShell 是最底层的系统级脚本环境。

然而，大语言模型在生成 PowerShell 命令行时，极易落入【双重 Shell 解析陷阱】（外层 Git Bash / POSIX 命令行解释器 -> 内层 powershell.exe -> 目标执行文件）。由此引发的引号剥离、特殊字符篡改与跨版本语法混淆，会导致命令连续报错与无效空转。

本指南提炼了智能体在自动化执行 PowerShell 命令时的典型报错根因、规避模式与黄金工程范式，作为 Agent 知识库（Knowledge）的基础事实支撑。

2. 核心陷阱与实战报错诊断根因表

根据大规模自动化日志实测，智能体编写 PowerShell 遇到 90% 以上的失败集中在以下 6 种模式：

[模式 1] 冒号语法解析错误 (ParserError: (:))
- 典型报错：ParserError: (:) / Unexpected token ':' in expression or statement
- 真实根因：在 -Command 中使用了 -f 格式化操作符（如 "{0}: {1}" -f $a, $b），在跨 Shell 传输中双引号被外层剥离，{0}: {1} 被误识别为非法 ScriptBlock
- 推荐正向写法：使用原生字符串内插："$($a): $($b)"

[模式 2] 版本特性泄漏 (PS 7+ vs PS 5.1)
- 典型报错：ParserError: (&&) / The token '&&' is not recognized in this version
- 真实根因：Windows 默认环境为 Windows PowerShell 5.1，模型训练数据中包含 PS 7+ 的管道链（&&、||）或三元运算符（? :），在 5.1 中无法解析
- 推荐正向写法：使用分号顺序执行：cmd1; if ($?) { cmd2 }，或纯粹单行独立命令

[模式 3] 等号与参数错位 (ObjectNotFound: (=Stop:String))
- 典型报错：ObjectNotFound: (=Stop:String) / (=path:String)
- 真实根因：将参数赋值写成了带等号的形式（如 -ErrorAction =Stop 或 $var = E:\test 未加引号），导致 '=' 被当做独立命令或参数名
- 推荐正向写法：正规参数传递：-ErrorAction Stop；变量赋值包裹引号：$var = 'E:\test'

[模式 4] Where-Object 简写歧义 (InvalidArgument: ... [Where-Object])
- 典型报错：InvalidArgument: ... [Where-Object], PSArgumentException
- 真实根因：使用无大括号简写形式（如 Where-Object Name -like *.pdb）遇到通配符或特殊属性时解析失败
- 推荐正向写法：强制使用标准脚本块：Where-Object { $_.Name -like '*.pdb' }

[模式 5] .NET 静态类调用异常 (InvalidOperation: (...:TypeName))
- 典型报错：InvalidOperation: (System.IO.File:TypeName) / (Console:TypeName)
- 真实根因：在双引号内插中直接书写 [System.IO.File]::ReadAllText($p)，方括号未与表达式正确结合
- 推荐正向写法：使用子表达式包裹：$($text = [System.IO.File]::ReadAllText($p))，或落盘脚本执行

[模式 6] Foreach 变量名缺失 (MissingVariableNameAfterForeach)
- 典型报错：ParserError: MissingVariableNameAfterForeach
- 真实根因：在单行命令中使用 foreach ($ in ...) 或跨 Shell 变量被 Bash 提前展开变为空
- 推荐正向写法：使用标准管道：... | ForEach-Object { ... }，或外层使用强单引号包裹

3. PowerShell 黄金工程范式

【范式 1：单行命令极简稳健原则 (Simple One-Liners)】
用于轻量级目录探查、文件过滤与只读检索：
1. 外层强引用包裹：在 Git Bash 等 POSIX 终端下，外层统一使用单引号包裹完整命令，防止 Bash 展开内部 $ 变量：
```
powershell.exe -NoProfile -Command '$root = "E:\test"; Get-ChildItem -Path $root'
```
2. 变量内插替代格式化：避免使用 -f 格式化操作符拼接字符串，全面采用原生子表达式变量内插：
```
$ast.FindAll({ param($n) $n -is [System.Management.Automation.Language.FunctionDefinitionAst] }, $true) | ForEach-Object { "$($_.Name): $($_.Extent.StartLineNumber)-$($_.Extent.EndLineNumber)" }
```

【范式 2：复杂多行逻辑强制走 -File script.ps1 模式 (File-First Paradigm)】
当满足以下任一条件时，停止在命令行中拼接复杂长字符串，转为生成独立的 .ps1 脚本文件执行：
1. 包含超过 2 行的条件分支或多重循环；
2. 包含 AST 抽象语法树解析（[System.Management.Automation.Language.Parser]）；
3. 包含复杂的正则替换、多管道处理或自定义类加载。

实施步骤：
1. 生成脚本文件：通过智能体原生文件生成或落盘工具，在 .cph/scratch/ 或工作区创建 inspect.ps1；
2. 免转义执行脚本：
powershell.exe -NoProfile -ExecutionPolicy Bypass -File ".cph/scratch/inspect.ps1"

核心优势：
- 零 Shell 引号剥离；
- 原始换行与语法 100% 保真；
- 输出异常时提供精准的文件与行号堆栈。

4. 常见任务推荐 Cmdlet 对照表

- 文件检索：Get-ChildItem
示例：Get-ChildItem -Path 'src' -Filter '*.ts' -Recurse | Select-Object FullName
- 内容检索：Select-String
示例：Select-String -Path 'src/*.ts' -Pattern 'export function' -CaseSensitive
- 安全读取文件：Get-Content
示例：Get-Content -Path 'file.txt' -TotalCount 100 或 Get-Content -Path 'file.txt' -Tail 50
- 文本落盘：Set-Content / Out-File
示例：Set-Content -Path 'out.txt' -Value $data -Encoding UTF8
- AST 语法解析：[System.Management.Automation.Language.Parser]::ParseFile
规范：统一在 .ps1 文件内编写并执行，避免命令行转义损毁。

5. 编码与本地化避坑规范

---

1. UTF-8 字符编码：

Windows 终端默认可能使用 ANSI / GBK 编码，导致输出包含 Unicode 字符（如 中文、箭头等）时报编码错误或出现乱码。在执行涉及非 ASCII 内容的命令时，显式配置控制台编码：

[Console]::OutputEncoding = [System.Text.Encoding]::UTF8

2. 路径分隔符稳健性：

PowerShell 原生兼容正斜杠 '/' 与反斜杠 '\'。推荐在命令中统一使用正斜杠（如 'E:/test/src/index.ts'），避免反斜杠在跨环境传输中被解释为转义字符。